International Islamic University Islamabad

Faculty of Engineering and Technology

Department of Electrical and Computer Engineering

Artificial Intelligence and Machine Learning CEP Report

Project Title: Intelligent Email Spam Detection Using Machine Learning and Natural Language Processing.

Name of Student: Muhammad Zain Tariq

Roll No.: 04-FET/BSCE/F22

Submitted on: 02-06-2025

INTERNATIONAL ISLAMIC UNIVERSITY ISLAMABAD
FACULTY OF ENGINEERING & TECHNOLOGY
DEPARTMENT OF ELECTRICAL & COMPUTER ENGINEERING
BS Computer Engineering

COURSE NAME	AI and ML	**BATCH**	F22
COURSE CODE	CE-418	**MAX. MARKS**	
INSTRUCTOR	Dr. Muhammad Sajjad Khan	**DEADLINE**	
Note: Instructions (if Any)			

Complex Engineering Problem (CEP)
Intelligent Email Spam Detection Using Machine Learning and Natural Language Processing

Problem Statement:

E-Mail remains a fundamental communication tool, but its efficiency is increasingly threatened by the increasing presence of spam messages. Modern spam is more refined than ever, manipulating both the content of the text and structural elements such as length, format, embedded connections, and capital to avoid traditional methods of identification. Instead of relying solely on obvious keywords, Spammer has become an emotionally persuasive term, including win, free, embedded malicious links and messages that tinker with messages that satisfy legal emails. Traditional rule-based filters often do not adapt to these misleading strategies. Therefore, there is a need for an intelligent spam recognition system that uses machine learning to analyze both the content and structure of the E-mail. The purpose of this project is to develop and evaluate such systems using natural language processing and several machine learning models to recognize spam.

Theoretical Basis:

This project focuses on **spam email classification** using Natural Language Processing (NLP) and Machine Learning (ML). At its core, the task is a **binary classification problem**: given an email, the system must predict whether it is **spam** or **ham (not spam)**.

Traditional rule-based filters struggle with modern spam emails that creatively avoid keyword-based detection. Hence, we adopt an **intelligent, data-driven approach** by analyzing both:

- **Textual content** using NLP
- **Structural patterns** (length, links, capitalization, etc.)

Working Flow of the Spam Detection System

1. **User Input**
    - The user enters or pastes an email message into the Gradio input interface.
2. **Text Preprocessing**
    - The input email is preprocessed automatically:
        - Converted to lowercase
        - Punctuation, URLs, and special characters removed

        - Stopwords removed using NLTK

3. **Feature Extraction**
    - The cleaned email is transformed using:
        - **TF-IDF Vectorizer** for textual features.
        - **Rules** for structural features (length, links, capital ratio, etc.)
4. **Model Prediction**
    - The combined feature vector is passed to the **trained Logistic Regression model**.
    - The model predicts whether the email is:
        - Spam
        - Ham (Not Spam)
5. **Confidence Level Output**
    - The model shows a **confidence score (in %)** indicating how sure it is about the prediction.
6. **Email Count Tracking**
    - The system automatically updates:
        - Total number of emails tested
        - Count of detected Spam vs Ham emails
7. **Real-Time Graph Update**
    - A **live bar chart** displays:
        - Number of Spam emails
        - Number of Ham emails
        - Total emails tested
8. **Output Displayed to User**
    - Final output shown on screen includes:
        - Prediction: Spam / Ham
        - Confidence Score
        - Updated Bar Chart (Spam vs Ham)
        - Total emails tested so far

Step-by-Step Project Pipeline

1. Dataset

- Source: Kaggle (190K+ labeled emails)
- Labels: ham (not spam), spam
- Fields: text, label.

```python
# 1) Data acquisition
import pandas as pd
data=pd.read_csv('spam_Emails_data.csv')
print(data.head())
print(data.columns)
print(data.isnull().sum())
print(data['label'].value_counts())
```

```
  label                                               text
0  Spam  viiiiiiagraaaa\nonly for the ones that want to...
1   Ham  got ice thought look az original message ice o...
2  Spam  yo ur wom an ne eds an escapenumber in ch ma n...
3  Spam  start increasing your odds of success & live s...
4   Ham  author jra date escapenumber escapenumber esca...
Index(['label', 'text'], dtype='object')
label    0
text     0
dtype: int64
label
Ham     1761
Spam    1577
Name: count, dtype: int64
```

2. Data Preprocessing

Steps:

- Lowercasing, punctuation removal
- Removing hyperlinks, special characters
- Stopwords removal (using NLTK)
- Handling missing/null values.

```python
[5]  # 2)  Cleaning the data (full cleaning)
     import re
     import pandas as pd
     import numpy as np

     def clean_text(text):
         # Check if the value is NaN or not a string
         if isinstance(text, float) and np.isnan(text):
             return ""

         # Convert to string if it's not already
         text = str(text)

         # Lowercase
         text = text.lower()
         # Remove URLs
         text = re.sub(r'http\S+|www.\S+', '', text)
         # Remove punctuations and numbers
         text = re.sub(r'[^a-z\s]', '', text)
         text = re.sub(r'http\S+|www.\S+', ' <LINK> ', text)
         # Remove extra spaces
         text = re.sub(r'\s+', ' ', text).strip()
         return text

     # Apply the function and handle NaN values
     data['clean_text'] = data['text'].apply(clean_text)
```

```
[6]  # Removing stop keywords
     import nltk
     from nltk.corpus import stopwords

     nltk.download('stopwords')
     stop_words = set(stopwords.words('english'))

     def remove_stopwords(text):
         return ' '.join([word for word in text.split() if word not in stop_words])

     data['clean_text'] = data['clean_text'].apply(remove_stopwords)
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

3. Feature Engineering

1. **Textual Feature:**
    - TF-IDF Vectorization (Top 5000 words)
2. **Structural Features:**
    - Email Length
    - Number of Links
    - Capital Ratio
    - Presence of Spammy Words (win, free, etc.)

```
[7]  # 3) Text feature extraction TF-IDF
     from sklearn.feature_extraction.text import TfidfVectorizer

     tfidf = TfidfVectorizer(max_features=5000)  # Take top 5000 most important words
     X_text = tfidf.fit_transform(data['clean_text']).toarray()
```

```
# 1) Email Length (more words = might be spam)
# Convert non-string entries to string or empty string
data['email_length'] = data['text'].fillna('').astype(str).apply(len)

# 2) Number of Links (spams often have lots of links)
import re

def count_links(text):
    if isinstance(text, str):
        return len(re.findall(r'http\S+', text))
    return 0

data['num_links'] = data['text'].apply(count_links)


# 3) Capital Letter Ratio (spams often SHOUT)
def capital_ratio(text):
    if not isinstance(text, str):
        text = str(text) if text is not None else ''
    capitals = sum(1 for c in text if c.isupper())
    return capitals / max(1, len(text))

data['capital_ratio'] = data['text'].apply(capital_ratio)

# 4) Presence of Spammy Words ("FREE", "WIN", "URGENT", etc.)
def has_spammy_words(text):
    spammy = ['free', 'win', 'guarantee', 'winner', 'urgent']
    text = str(text).lower()
    return int(any(word in text for word in spammy))

data['spammy_words'] = data['text'].apply(has_spammy_words)
```

[9]
```
# Now Merging all 4 features
import numpy as np

# Ensure no NaNs in structural features : safe handling data
data[['email_length', 'num_links', 'capital_ratio', 'spammy_words']] = \
    data[['email_length', 'num_links', 'capital_ratio', 'spammy_words']].fillna(0)

# Extract structural features as a NumPy array
X_structural = data[['email_length', 'num_links', 'capital_ratio', 'spammy_words']].values

# Merge TF-IDF text features with structural features
X_final = np.hstack((X_text, X_structural))   # complete feature set for machine learning features + all data
```

```
[10] # prepare labels
     y_final = data['label'].map({'ham': 0, 'spam': 1})  # Map ham -> 0, spam -> 1
```

```
[11] #print(y_final)
     print(data['label'].isna().sum())  # This will show the number of NaN values in your labels column
```

```
0
```

```
[12] # print(X_final.shape)
     # print(data['label'].shape)  # Check the shape of the label column
     # Remove rows where 'label' column has NaN values

     # Remove rows where 'label' is NaN
     data = data.dropna(subset=['label'])

     # Reset the index to keep things clean (optional but recommended)
     data = data.reset_index(drop=True)

     # Now extract labels
     y_final = data['label'].values
```

```
[13] # check if any NaNs remain
     print(data['label'].isnull().sum())  # Correct way for object/string types
```

```
0
```

```
[15] # check shape of X and y final
     print("X_final shape:", X_final.shape)
     print("y_final shape:", y_final.shape)
```

```
X_final shape: (3338, 5004)
y_final shape: (3338,)
```

```
[16] # cheching features of first mail
     print("First email features (X_final):", X_final[0])
```

```
First email features (X_final): [0. 0. 0. ... 0. 0. 0.]
```

```
[17] print(data['label'].unique())
     data['label'] = data['label'].str.strip().str.lower()
     y_final = data['label'].map({'ham': 0, 'spam': 1})
```

```
['Spam' 'Ham']
```

```
[18] # check label values
     print("First 10 labels:", y_final[:10].values)
```

```
First 10 labels: [1 0 1 1 0 1 0 0 0 0]
```

```
[19] print(y_final.value_counts())
```

```
label
0    1761
1    1577
Name: count, dtype: int64
```

```
[20] print(X_final[10])
```

```
[0. 0. 0. ... 0. 0. 1.]
```

```
[21] print(data['clean_text'])
```

```
0       viiiiiiagraaaa ones want make scream prodigy s...
1       got ice thought look az original message ice o...
2       yo ur wom ne eds escapenumber ch n b e th n f ...
3       start increasing odds success live sexually he...
4       author jra date escapenumber escapenumber esca...
                              ...
3333    original message jamey estes jason dobbs cc oo...
3334    srea takes investors second climb escapenumber...
3335               escapelong dynpzfgtf x kkxgxz xhwro wq
3336    greetings amazon com weve recently learned sup...
3337    bodylabel subject great parttime summer job di...
Name: clean_text, Length: 3338, dtype: object
```

```
[22] # saving cleaned data after data preprocessing
     data.to_csv('cleaned_dataset.csv', index=False)
```

4. Model Training

Training/Testing Split: using 80:20 principle 80% Training and 20% Training.

```
[19] # Testing training Split
     from sklearn.model_selection import train_test_split

     X_train, X_test, y_train, y_test = train_test_split(X_final, y_final, test_size=0.2, random_state=42)
     # 80% training 20% testing
```

Algorithms Used:

1. **Naive Bayes**

```
# 4) Now Training ML Models

# 1) Naive Bayes
from sklearn.naive_bayes import MultinomialNB

nb_model = MultinomialNB()
nb_model.fit(X_train, y_train)
```

```
MultinomialNB
MultinomialNB()
```

2. **Logistic Regression** ***(Best model)***

```
[25] # 2) Logistic Regression
     from sklearn.linear_model import LogisticRegression

     lr_model = LogisticRegression(max_iter=1000)
     lr_model.fit(X_train, y_train)
```

```
LogisticRegression
LogisticRegression(max_iter=1000)
```

3. Random Forest

```
[26] # 3)  Random Forest
     from sklearn.ensemble import RandomForestClassifier
     rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
     rf_model.fit(X_train, y_train)
```

```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

5. Model Evaluation

Metrics Used:

- Precision
- Recall
- F1-Score
- Confusion Matrix
- Accuracy

```python
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# List of models to compare
models = [nb_model, lr_model, rf_model]
names = ['Naive Bayes', 'Logistic Regression', 'Random Forest']

# Track best model
best_model = None
best_f1 = 0
best_name = ""
best_y_pred = None

print("Comparing Models...\n")

# Evaluate each model
for model, name in zip(models, names):
    y_pred = model.predict(X_test)

    report = classification_report(y_test, y_pred, output_dict=True)
    f1_spam = report['1']['f1-score']

    print(f"  {name}")
    print(f"F1-Score (Spam): {f1_spam:.4f}")
    print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
    print("Confusion Matrix:")
    print(confusion_matrix(y_test, y_pred))
    print()

    # Update best model based on spam F1-score
    if f1_spam > best_f1:
        best_f1 = f1_spam
        best_model = model
        best_name = name
        best_y_pred = y_pred

# Final best model results
print("Final Results for Best Model:", best_name)
print(f"Accuracy: {accuracy_score(y_test, best_y_pred):.4f}")
print("Classification Report:")
print(classification_report(y_test, best_y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, best_y_pred))
```

```
Comparing Models...

  Naive Bayes
F1-Score (Spam): 0.8170
Accuracy: 0.8166
Confusion Matrix:
[[1573  478]
 [ 231 1583]]

  Logistic Regression
F1-Score (Spam): 0.9572
Accuracy: 0.9594
Confusion Matrix:
[[1953   98]
 [  59 1755]]

  Random Forest
F1-Score (Spam): 0.9567
Accuracy: 0.9594
Confusion Matrix:
[[1973   78]
 [  79 1735]]
```

```
import joblib

# Save trained model
joblib.dump(lr_model, "spam_classifier.pkl")

# Also save TF-IDF vectorizer if needed
joblib.dump(tfidf, "tfidf_vectorizer.pkl")
```

```
['tfidf_vectorizer.pkl']
```

```
[29] import joblib
     model = joblib.load("spam_classifier.pkl")  # Your saved Logistic Regression
```

Best Model Selected

Logistic Regression

- Used TF-IDF + Structural features
- Balanced training (class_weight='balanced')

```
Final Results for Best Model: Logistic Regression
Accuracy: 0.9594
Classification Report:
              precision    recall  f1-score   support

           0       0.97      0.95      0.96      2051
           1       0.95      0.97      0.96      1814

    accuracy                           0.96      3865
   macro avg       0.96      0.96      0.96      3865
weighted avg       0.96      0.96      0.96      3865

Confusion Matrix:
[[1953   98]
 [  59 1755]]
```

Deployment with Gradio

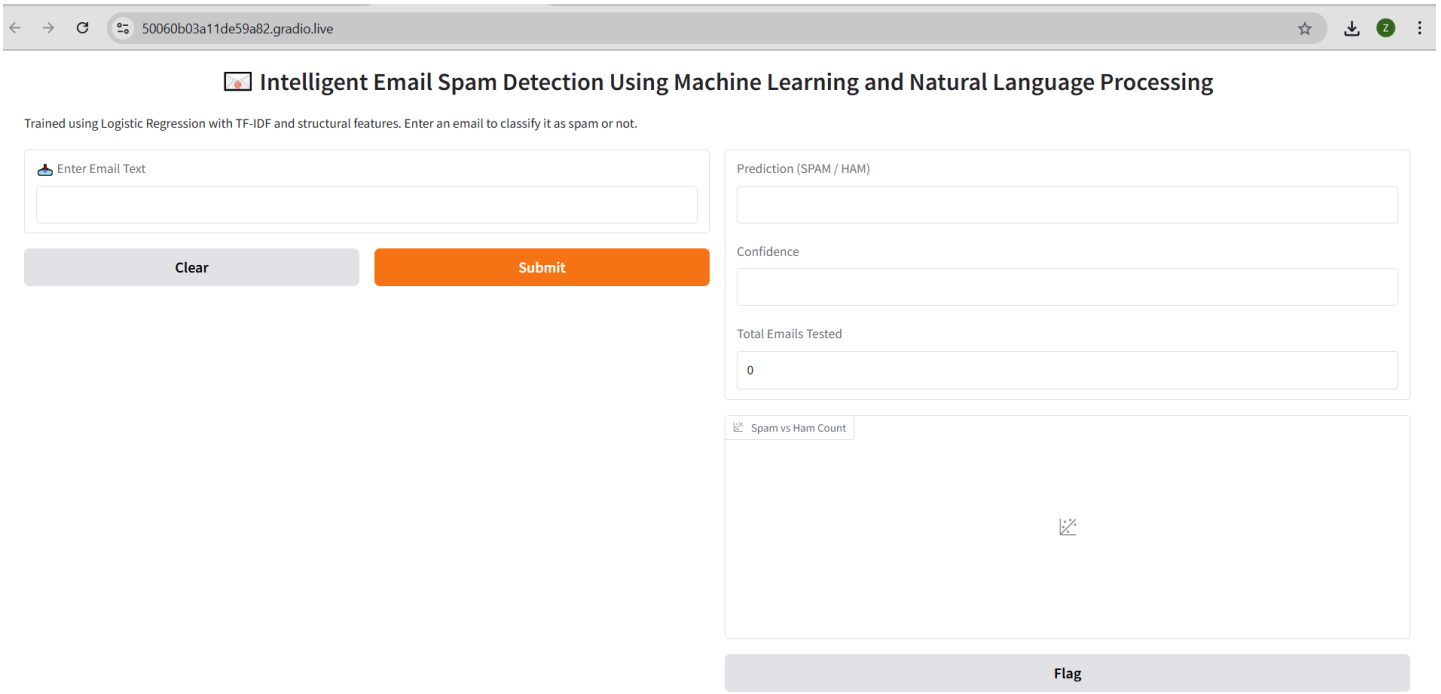
- Model + TF-IDF vectorizer saved via joblib
- Live predictions using gr.Interface()
- Confidence percentage shown
- Visualization: Bar chart of Spam vs Ham count (live updates)

```python
import gradio as gr
import joblib
import numpy as np
import matplotlib.pyplot as plt
import re

# Load best model (Logistic Regression) and vectorizer
model = joblib.load("spam_classifier.pkl")
vectorizer = joblib.load("tfidf_vectorizer.pkl")

# Global counters
email_count = 0
spam_count = 0
ham_count = 0

# === Preprocessing functions ===

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"http\S+|www.\S+", "", text)  # remove links
    text = re.sub(r"[^a-z\s]", "", text)         # remove punctuation/numbers
    text = re.sub(r"\s+", " ", text).strip()     # remove extra spaces
    return text

def has_spammy_words(text):
    spam_words = ["free", "win", "guarantee", "urgent", "winner", "click", "offer", "money"]
    return int(any(word in text.lower() for word in spam_words))

def count_links(text):
    if isinstance(text, str):
        return len(re.findall(r'http\S+', text))
    return 0
```

```python
def capital_ratio(text):
    caps = sum(1 for c in text if c.isupper())
    return caps / max(1, len(str(text)))

def get_structural_features(text):
    return np.array([
        len(text),
        count_links(text),
        capital_ratio(text),
        has_spammy_words(text)
    ]).reshape(1, -1)

# === Main prediction function ===

def predict_email(email):
    global email_count, spam_count, ham_count

    email_count += 1

    cleaned = clean_text(email)
    tfidf_vector = vectorizer.transform([cleaned]).toarray()
    struct_features = get_structural_features(email)
    final_input = np.hstack((tfidf_vector, struct_features))

    pred = model.predict(final_input)[0]
    proba = model.predict_proba(final_input)[0][pred]

    label = "SPAM" if pred == 1 else "HAM"
    confidence = f"{proba * 100:.2f}%"

    if pred == 1:
        spam_count += 1
    else:
        ham_count += 1
```

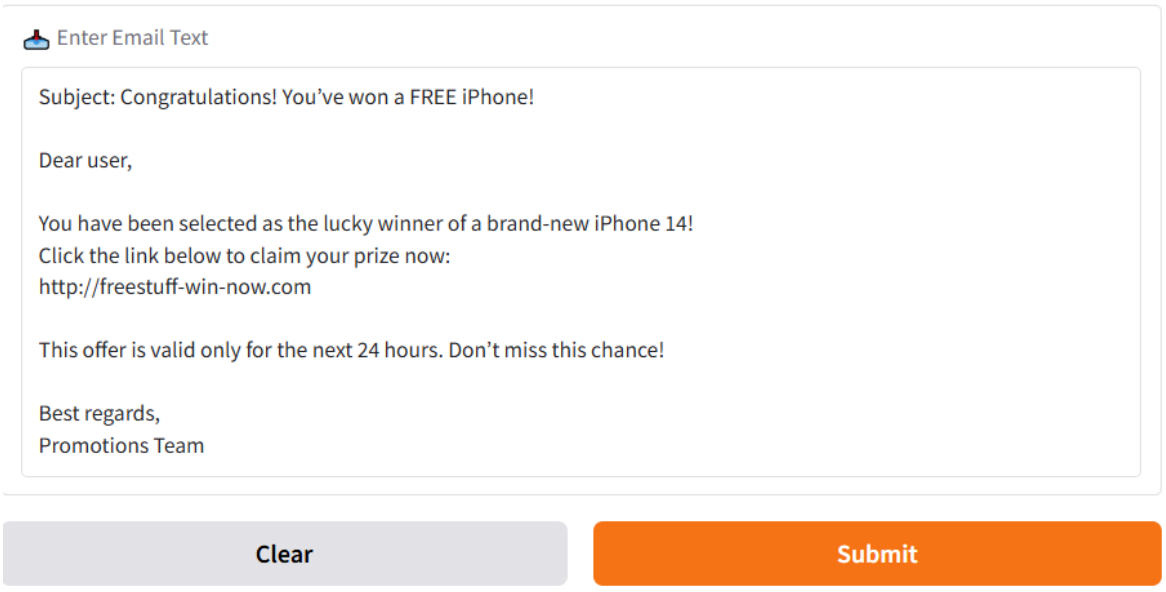
```python
    # Create live bar chart
    fig, ax = plt.subplots()
    ax.bar(["Spam", "Ham"], [spam_count, ham_count], color=["red", "green"])
    ax.set_title("Spam vs Ham Email Count")
    ax.set_ylabel("Emails")
    plt.tight_layout()
```

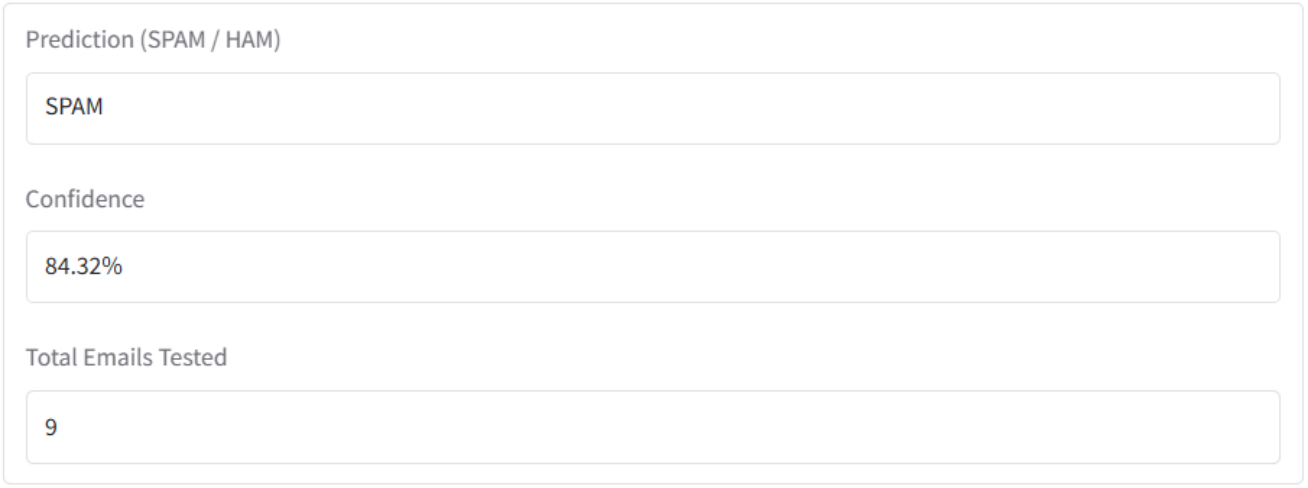
```python
# === Gradio UI ===

interface = gr.Interface(
    fn=predict_email,
    inputs=gr.Textbox(label="📥 Enter Email Text"),
    outputs=[
        gr.Textbox(label="Prediction (SPAM / HAM)"),
        gr.Textbox(label="Confidence"),
        gr.Number(label="Total Emails Tested"),
        gr.Plot(label="Spam vs Ham Count")
    ],
    title="📩 Intelligent Email Spam Detection Using Machine Learning and Natural Language Processing",
    description="Trained using Logistic Regression with TF-IDF and structural features. Enter an email to classify it as spam or not."
)
interface.launch()
```



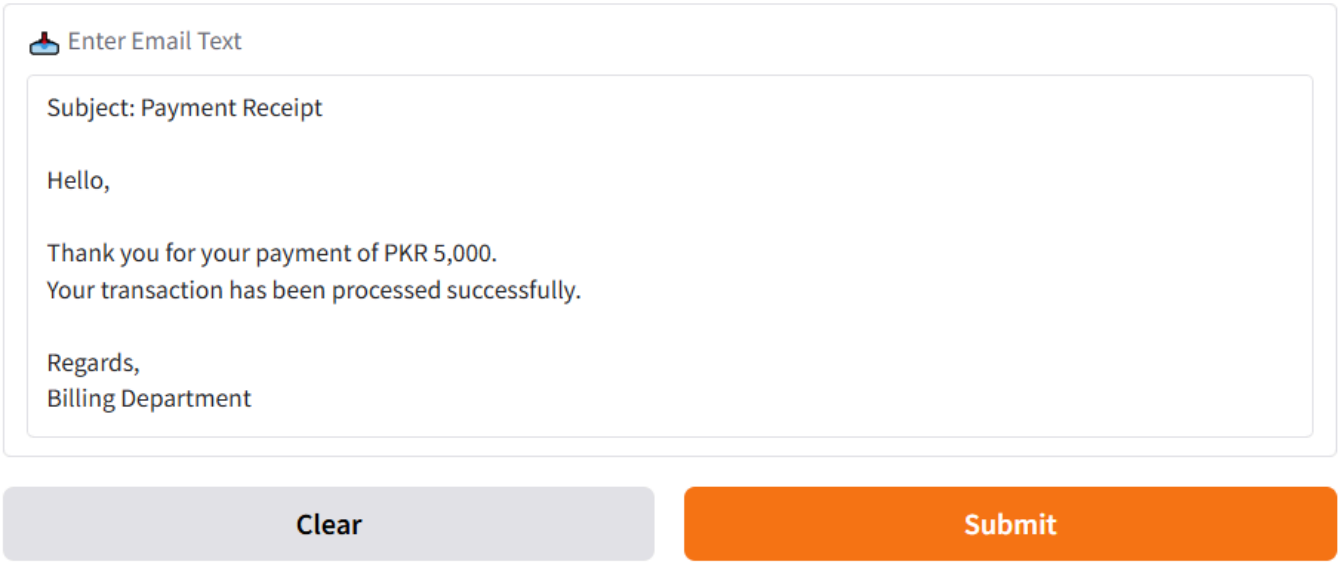
Sample Email Tests

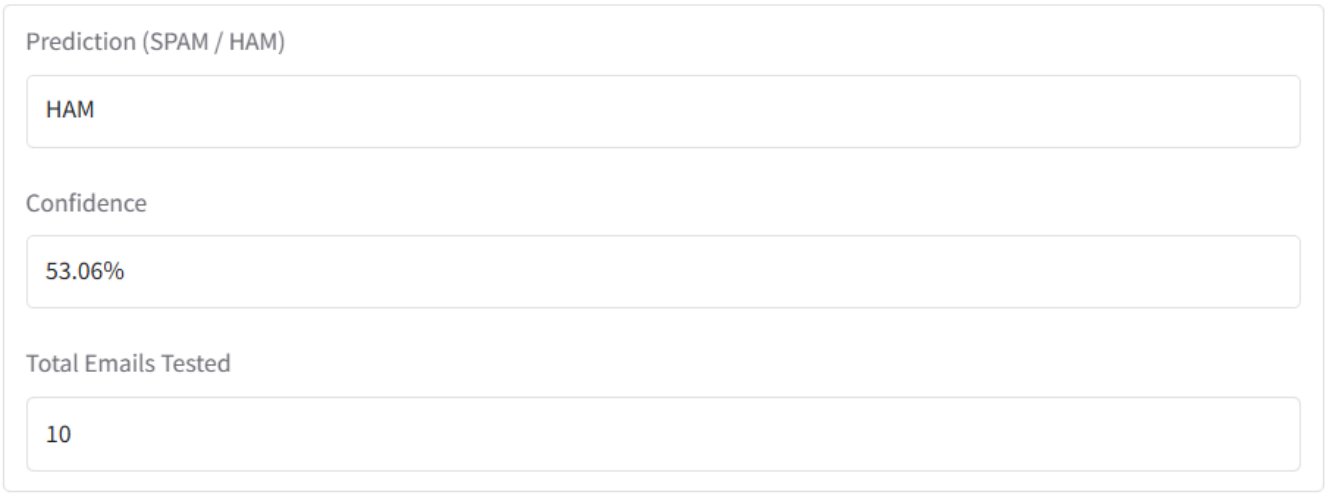
Test 1 :



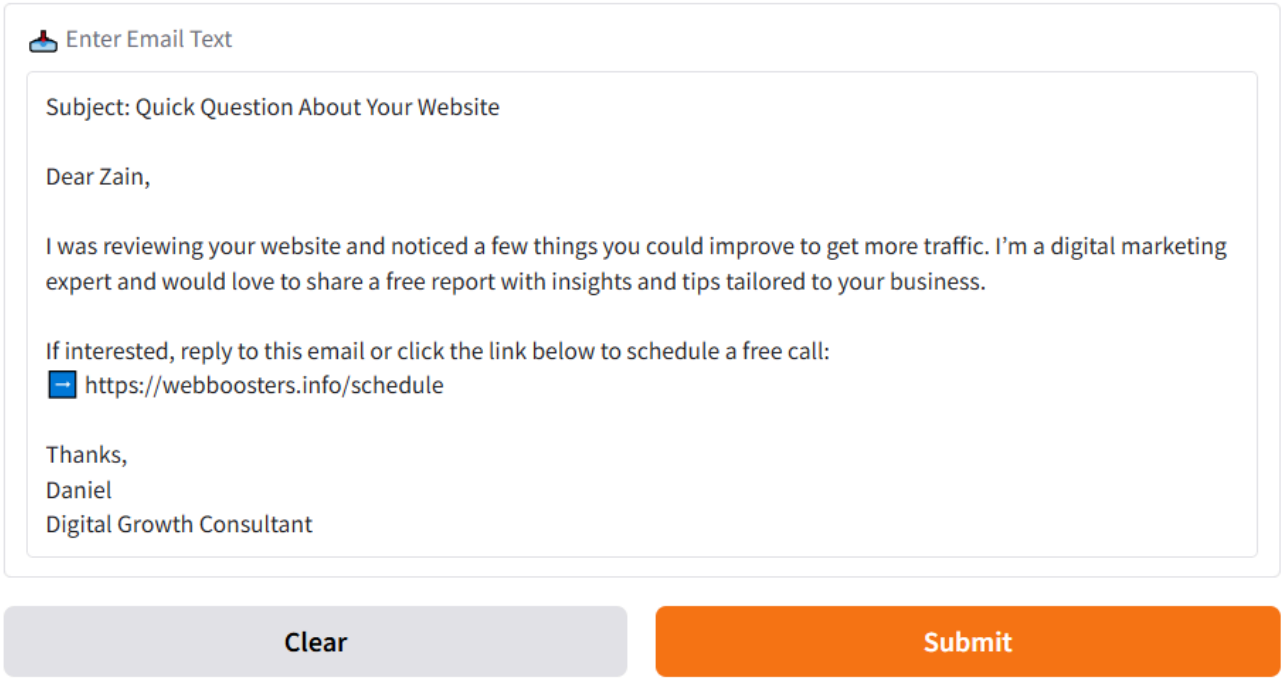

Prediction (SPAM / HAM)
SPAM
Confidence
84.32%
Total Emails Tested
9

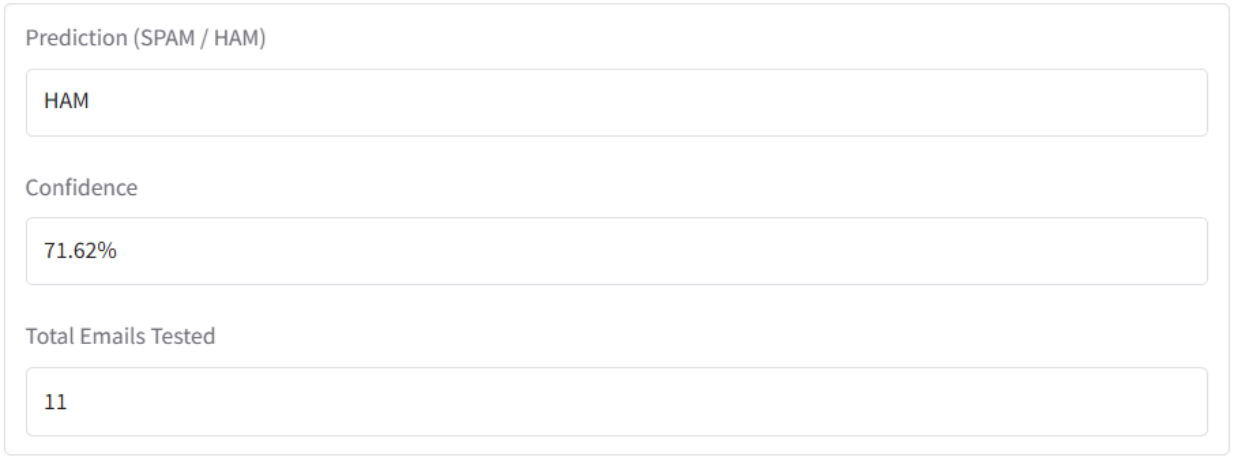
Test 2:


Enter Email Text
Subject: Payment Receipt
Hello,
Thank you for your payment of PKR 5,000.
Your transaction has been processed successfully.
Regards,
Billing Department
Clear
Submit


Prediction (SPAM / HAM)
HAM
Confidence
53.06%
Total Emails Tested
10

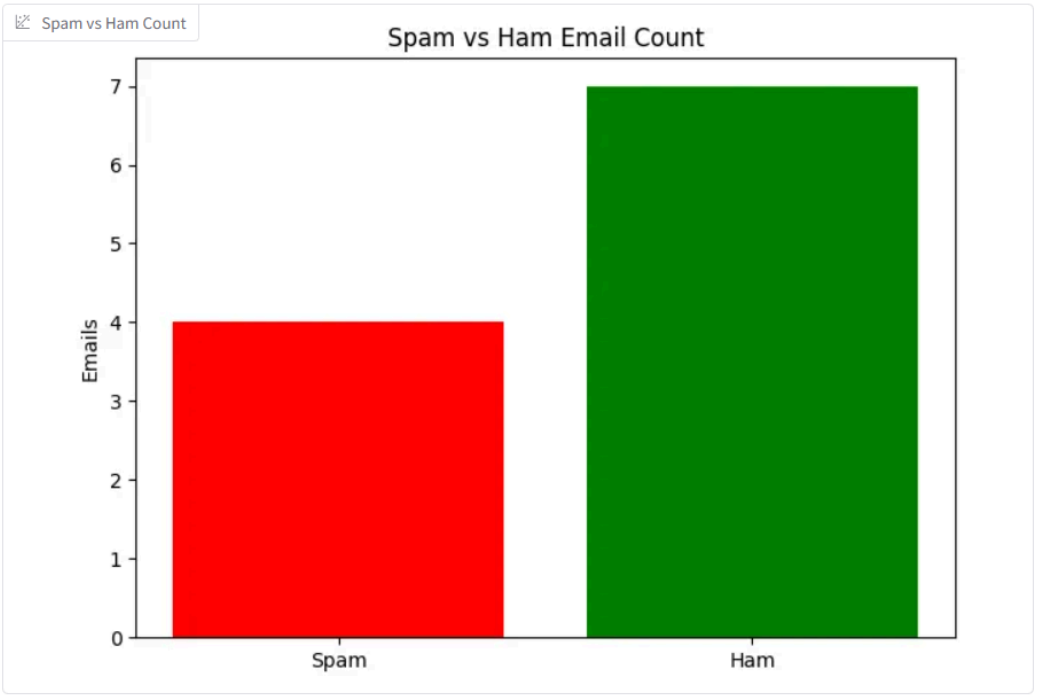
Test 3:





Visualizations

- Spam vs Ham Email Count (live bar chart)
- Feature importance (optional)
- Distribution of email lengths, link counts



Flag

Tools & Technologies Used

- Language: Python
- Libraries: Pandas, NumPy, Scikit-learn, Matplotlib, NLTK, Gradio
- ML Models: Logistic Regression, Naive Bayes, Random Forest
- Deployment: Gradio (on Google Colab)
- Preprocessing: TF-IDF + Regex + NLP
- Data Visualization: Matplotlib, Seaborn

Results & Conclusion

- Logistic Regression achieved the best performance with 95.94% accuracy and excellent spam recall.
- Structural features boosted performance, especially for tricky spam.
- Gradio deployment enabled real-time interactive predictions.

Future Work

- Add email subject header, sender features
- Use deep learning (RNN/LSTM or BERT) for contextual analysis
- Integrate with Gmail/Outlook APIs
- Build email filter plugin

References

1. Scikit-learn Documentation
2. NLTK Stopwords
3. Gradio for ML UI
4. Kaggle Email Spam Dataset
5. Research Papers on Spam Detection using NLP